



Corso di Laurea Triennale in Informatica

Crittografia visuale e calcolo sicuro:

Implementazione in Python del protocollo V2PC

RELATORE

Prof. Roberto De Prisco

CANDIDATO

Gennaro Carmine Tozza

Matricola 0512120382

Anno Accademico 2025/2026

Scansiona per dare
uno sguardo alla tesi →



Struttura della presentazione

01

Il problema

Calcolo sicuro a due parti

02

Gli strumenti

Crittografia visuale e circuiti garbled

03

Il protocollo V2PC

Porta visuale e fasi del protocollo

04

Il lavoro svolto

Libreria Python, demo web e stampa



Il problema

Alice e Bob

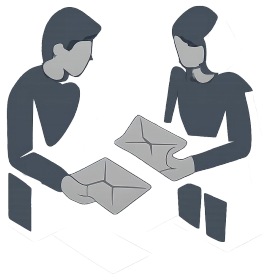
Vogliono calcolare $f(x, y)$ senza rivelare i propri input: ciascuno impara solo il risultato.

Origine (Yao, anni '80)

Il problema del “chi è più ricco”:
confrontare valori senza rivelarli.

Il limite

Le soluzioni esistenti presuppongono computer, chiavi e fiducia nella macchina.



Crittografia visuale

NAOR E SHAMIR, 1994

Il segreto è diviso in **share**: ciascuna da sola è rumore, sovrapposte rivelano l'immagine.



Segreto



Share 1



Share 2



Sovrapposizione

Ricostruzione

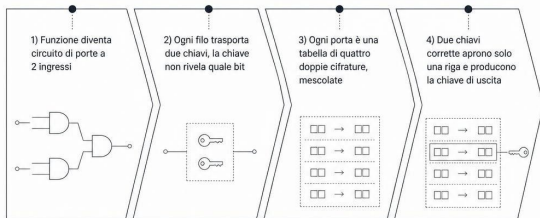
La fa l'occhio: nessun calcolo.

Sicurezza

Incondizionata, come il One-Time Pad.



I circuiti garbled di Yao



Struttura della porta

Ogni porta è una tabella di **quattro doppie cifrature**, mescolate.

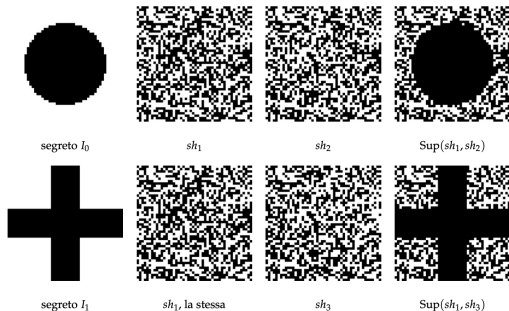
Le due chiavi giuste aprono una sola riga e restituiscono la chiave d'uscita.

$$\text{Enc}_{k_a^{\omega_i}}(\text{Enc}_{k_b^{\omega_j}}(k_{G(a,b)}^{\omega_k}))$$

Una riga per ogni coppia (a, b) : ω_i, ω_j ingressi, ω_k uscita.



Una share, due segreti



Tre share, due coalizioni

La share sh_1 è la stessa trasparenza, ma combinata con share diverse rivela due immagini diverse:

$$(sh_1, sh_2) \rightarrow I_0 \quad (sh_1, sh_3) \rightarrow I_1$$

Nessuna espansione dei pixel: le share sono grandi come il segreto.

Il nero torna esatto, il bianco appare grigio: è il prezzo del random grid.



Dalla porta cifrata alla porta visuale



$sh_{1,0}$ (sinistro, 0) $sh_{1,1}$ (sinistro, 1)

$sh_{2,0}$ (destro, 0)

$sh_{2,1}$ (destro, 1)

pointer:  $p = 0$

pointer:  $p = 1$

sul filo destro non compare un pointer in chiaro

Valori come immagini

Bianca per 0, nera per 1.
Due istanze, A e B , una
per valore del filo sinistro.

Valutazione

Chi valuta **ritaglia una
metà** della share destra e
la sovrappone.

Pointer bit

$p = v_1 \oplus b$, con b casuale:
indica la metà **senza rive-
lare il valore del filo**.



Le due fasi del protocollo

1. Costruzione - Alice, con un computer

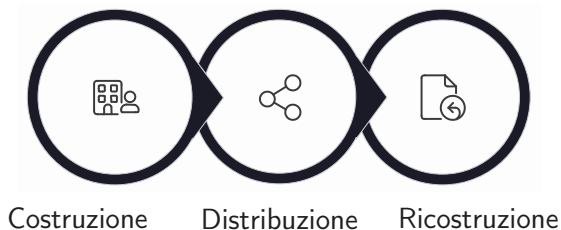
Dall'uscita verso gli ingressi: ogni porta produce le share dei propri due ingressi. Alla fine, ogni filo di ingresso ha due trasparenze, una per valore.

2. Valutazione - Forbici e lucidi

Alice consegna le proprie share. Per i bit di Bob avviene un **oblivious transfer fisico**. Bob risale il circuito: legge il pointer, taglia, sovrappone.



L'implementazione in Python



Caratteristiche

- Libreria Python scritta da zero
- Costruzione indipendente dagli input
- Ricostruzione dalle sole share selezionate
- Riga di comando, demo web e kit di stampa A4

Formule supportate

AND, OR, XOR e NOT, con variabili di Alice (x_i) e di Bob (y_i). Una variabile può ripetersi (non read-once).



La demo web e il kit di stampa

Configurazione dell'esperimento

FUNZIONE BOOLEANA

$(x1|y1) \& ((x2\&y2) \& (x3|y3))$

Operatori: &, AND, | OR, ^ XOR, ~ NOT. Le variabili che iniziano per x sono di Alice, quelle per y di Bob.

VALORI DELLE VARIABILI

Selezione 0 oppure 1

x1: 0 1 y1: 0 1 x2: 0 1 y2: 0 1 x3: 0 1 y3: 0 1

LATO IMMAGINE: 32

SEME (OPZIONALE): casuale

Simula il trasferimento e ricostruisci Scarica tutte le alternative

Risultato dell'esempio

Risultato



1

Coincide con il valore booleano

Valore vero	1
Porte	5
Profondità	3

Kit di stampa

- PDF A4 con scala comune
- Crocini di taglio e allineamento
- Valutazione a mano su lucidi



Risultati e misure

Esempio del paper: 5 porte, 3 livelli ($d = 2$), lato $s = 8$, uscita attesa 0.

Stima per profondità

0,02%

$$k = d + 1 = 3$$

Misurato

13,0%

391 errori su 3 000

Modello per porte

13,1%

$k = 5$ porte

$$P_{\text{err}} = (1 - 2^{-k})^{s^2} \quad k: \text{griglie casuali sovrapposte su ogni pixel}$$

Nella costruzione implementata k è il **numero di porte**: il lato passa da $s = 8$ a $s = 17$



Conclusioni

Contributo realizzato

Implementazione Python del V2PC, eseguibile in demo e riproducibile su carta.

Risultato delle misure

Sui circuiti ramificati l'errore segue il numero di porte, non la profondità: il V2PC resta per circuiti piccoli.

Limiti e sviluppi

L'oblivious transfer è simulato in locale.
Prossimi passi: OT fra due parti separate e scelta automatica del lato s .

Crittografia visuale
e calcolo sicuro

Grazie!

Gennaro Carmine Tozza

g.tozza1@studenti.unisa.it 

[gennarocarmine.github.io](https://github.com/gennarocarmine) 

[linkedin.com/in/gennaro-carmine-tozza](https://www.linkedin.com/in/gennaro-carmine-tozza) 